
Towards Perfect Code Generation: Iterative API-based Feedback Loops

Harman Preet Singh

852863316

Abstract

The abstract paragraph should be indented ½ inch (3 picas) on both the left- and right-hand margins. Use 10 point type, with a vertical spacing (leading) of 11 points. The word **Abstract** must be centered, bold, and in point size 12. Two line spaces precede the abstract. The abstract must be limited to one paragraph.

Keywords: keyword 1 · keyword 2 · keyword 3

1 Introduction (1 – 1.5 pages)

Large Language Models (LLMs) are increasingly used in the field of software development for the generation of code, including HTML. These models are very capable of producing syntactically valid HTML that renders correctly in web browsers. However, there is a significant blind spot: while the generated code may be syntactically correct, it often lacks semantic correctness. In other words, the code functions as intended, but it may not adhere to best practices.

This potential non-adherence to semantic best practices is often seen in the overuse of the `<div>` tag, a phenomenon colloquially referred to as "divitis". Instead of using semantically meaningful tags in layout and content structure, such as `<section>` to denote sections or `<article>` for articles, developers may default to using `<div>` tags. For example, a developer might wrap an entire article in a `<div>` instead of using the more appropriate `<article>` tag. LLMs are trained on real code snippets, which often contain this pattern, making them likely to reproduce it in their generated output. Additionally, if a codebase already contains a significant amount of "divitis" code-generation tools like Claude Code may produce new code that perpetuates this issue.

There are two real consequences. Code that is hard to maintain, as the semantic meaning of the content has become an afterthought. But more importantly, users with visual impairments who rely on screen readers are negatively affected because screen readers use the semantic structure of a webpage to navigate them. A lack of semantic structure can make it difficult for these users to understand the content and context of a page.

Instructing an LLM to use semantic tags through a system prompt or explicit instructions within the prompt itself does not guarantee compliance, particularly for smaller or local models. There is a need for a reliable automated mechanism that can catch and correct this class of semantic errors in LLM-generated HTML.

We propose a solution in the form of an iterative validation-and-regeneration feedback loop. In this approach, the LLM-generated HTML is first validated using a validator tool which can flag semantic issues. The LLM is then reprompted with the identified errors, allowing it to correct its output. This loop continues until the output is free of errors or a maximum number of iterations is reached.

In this report, we aim to find out (RQ1) to what extent an iterative feedback loop can improve the semantic correctness of LLM-generated HTML code, and (RQ2) whether this guardrail approach can enable smaller, cheaper LLMs to achieve semantic correctness comparable to larger models. The remainder of this report is structured as follows: Section 2 reviews related work, Section 3 presents the research questions and hypotheses, Section 4 details the methodology, Section 5 describes the experimental setup, Section 6 presents the results, and Section 7 discusses the findings and implications. Finally, we conclude with a summary of our contributions and potential future work.

2 Related Work (2 – 3 pages)

Related work spans two interconnected areas: (1) LLM code generation quality, and (2) iterative self-correction and feedback loops. In the first part, we discuss what the literature says about LLMs generating code: their strengths, but also the documented tendency to reproduce patterns from training data including bad practices. In the second part, we cover approaches where LLMs are given feedback and asked to revise their output, particularly for Answer Set Programming (ASP) code. Finally, we review what is known about HTML semantic quality as a measurable property and the use of automated validation as a guardrail.

The literature on LLM code generation highlights both the strengths and limitations of these models. While LLMs can autonomously generate functioning code, they are also prone to reproducing patterns from their training data, including suboptimal or even harmful coding practices. For instance, Souly et al. [1] demonstrate that LLMs can be susceptible to poisoning attacks, where maliciously crafted inputs lead to the generation of flawed code. They show that even a small amount of poisoned data can significantly degrade the model’s performance.

Tambon et al. [2] examined samples of 333 bugs collected from code generated using three leading LLMs and identified 10 common bug patterns, not including our focus on semantic quality. Dou et al. [3] evaluated several models on the length and complexity of generated code, finding that LLMs face challenges in generating successful code for more complex problems. They propose a novel training-free iterative method where the model gains the ability to self-critique and correct its own code, resulting in a repair success rate of 29.2% after two iterations. This study particularly is very relevant to our work, as it demonstrates the potential of iterative feedback loops in improving code quality. However, it primarily focuses on functional correctness rather than semantic or stylistic correctness, which is a gap that our research aims to address.

Addressing the second area of related work, several studies have explored iterative feedback and self-correction loops for LLMs. Ravi et al. [4] present a method that employs multiple feedback models to iteratively refine code generated by LLMs. Their approach demonstrates that iterative feedback can significantly enhance code quality, but it also introduces potential biases from the feedback models themselves. In contrast, our approach does not use other LLMs as feedback sources; instead, we employ an external, deterministic validator to provide feedback. This choice mitigates the risk of introducing additional biases and allows for a more objective assessment of code quality.

Madaan et al. [5] introduce ‘Self-Refine’, a self-refinement framework where the model critiques its own output and iteratively improves it. They tested the framework out on 7 diverse tasks, ranging from dialog response generation to mathematical reasoning, and showed that the generated code improves by 20% on average in task performance. The proposed method uses a single LLM as the generator, refiner and the feedback provider, whereas our approach separates the feedback mechanism from the generation process.

Much research has been conducted comparing the performance of smaller versus larger LLMs on code generation tasks. Coignon et al. [6] show that larger models generally outperform smaller ones in terms of code quality and functional correctness. However, they note that actual runtime execution efficiency of the generated code does not vastly differ, that is, smaller models can still produce code that runs efficiently.

Historically, treating semantic correctness as a measureable, objective property has been difficult because ‘meaning’ is highly contextual. However, some prior work has attempted to quantify semantic correctness in code. One example is the ‘Generic-to-Semantic Ratio’ (R_{gs}), which measures the count of non-semantic structural nodes (`<div>`, `<span>`) against native layout elements (`<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`). A lower ratio indicates higher structural semantic quality. The ‘Heading Hierarchy Traversal’ (H_{score}) method measures structural correctness by scanning a webpage for skipped heading levels (e.g., a jump from `<h1>` directly to `<h3>`, skipping `<h2>`). These approaches provide a way to quantify semantic quality, but they are limited in scope and do not capture the full semantic intent of the code. In comparison, the W3C Markup Validation Service is a widely used tool that checks for syntactic and structural conformance to HTML standards. It is capable of identifying errors such as unclosed tags, incorrect nesting, and invalid attributes. However, it does not assess the semantic intent of the code, which is a critical aspect of web accessibility and user experience. This gap highlights the need for more comprehensive evaluation methods that can capture both syntactic correctness and semantic quality in HTML code. Our work does not address this gap, instead we acknowledge the need for further research in this area.

No prior work combines an external deterministic validator, an iterative reprompting loop, and a cost-efficiency comparison across model sizes in the context of HTML semantic quality. Our research aims to fill this gap by providing a comprehensive evaluation of how iterative feedback loops can improve the semantic quality of code generated by LLMs. By using an external validator, we can objectively assess the quality of the generated code without introducing additional biases. Additionally, our study will explore the cost-efficiency of using smaller models in comparison to larger ones.

3 Research Questions & Hypotheses (0.5 pages)

We aim to answer two research questions:

1. **RQ1:** Does the iterative API-based feedback loop improve the quality of code generation? We operationalise "quality" as the reduction in the number of errors in the generated code, measured before and after applying the feedback loop.
2. **RQ2:** Does this guardrail approach enable smaller, cheaper LLMs to achieve comparable quality to larger, more expensive models? We define "comparable quality" as equivalent error reduction rates, and "cost" is measured via mean generation time per prompt per model. Cloud-based models are explicitly out of scope for this study, as all models are run locally through Ollama, ensuring consistent evaluation conditions and avoiding external dependencies.

To evaluate our feedback loop, we also test the models under a blind condition, where the regeneration is performed without detailed feedback. This allows us to isolate the effect of the feedback loop.

We expect the feedback condition to outperform the blind condition across all models, as the iterative feedback loop provides targeted guidance for error correction. Furthermore, we hypothesise that reasoning-capable models (i.e., models that think and reason through the problem before generating code) will converge faster regardless of parameter count. Additionally, we expect that difficult prompts will require more iterations to achieve error-free code generation.

4 Methodology (4 – 5 pages)

4.1 System Architecture

Describe the three components (LLM, validator, reprompting pipeline) and how they interact. Include your flowchart (Figure A1). Explain design choices, e.g. why Docker-hosted W3C rather than the public API.

4.2 Prompt Dataset

Describe how you constructed your prompt set. How many prompts? How did you decide which ones to include? Did you deliberately design some to provoke semantic tag errors (e.g. tasks that structurally require `<nav>`, `<article>`, etc.)? This directly addresses the supervisor's feedback about the random prompt selection being unclear.

Change: Remove the mention of random sampling — the dataset is now a fixed, ordered set of 51 prompts across three difficulty tiers (simple / medium / difficult). Describe how prompts were designed across tiers, and that the fixed ordering ensures apples-to-apples model comparisons.

4.3 Models & Configurations

List all models tested. Describe the parameter ranges (size, quantisation). Explain how you vary the maximum iteration count and how you determine when you've found the "optimal" cut-off — frame this as a quality-vs-cost tradeoff that you measure empirically.

Change: Expand this significantly. You now have a concrete model table to discuss (gemma4:e2b, qwen3:8b, qwen2.5-coder:14b, qwen3.5:9b) with an interesting emerging finding: reasoning capability matters more than parameter count. qwen2.5-coder:14b has more parameters than the others combined, yet it fails to fix issues because it lacks reasoning. This is now a key variable alongside model size. Add `temperature` and deterministically derived `seed` as explicit configuration parameters worth mentioning for reproducibility.

4.4 Validation Strategy

This is the most critical methodological section. Address the supervisor's concern directly: can the W3C validator actually detect semantic misuse of `<div>`? If it cannot (likely the case — W3C validates syntax, not semantics), you need to be transparent about this limitation and introduce a complementary evaluation. Options to discuss:

- A human evaluation rubric (e.g. annotators scoring semantic correctness)
- A secondary automated check (e.g. counting occurrences of semantic tags vs. `<div>` before and after)
- Possibly a separate LLM-as-judge evaluation

Clarify what "comparable output quality" means for RQ2 — define your quality metric explicitly.

Change: Add a subsection on the blind ablation control. This is a significant addition — for each run, the pipeline now runs both a `feedback` condition (model sees validator errors) and a `blind` condition (model is only told to "review and fix"). This lets you isolate how much improvement comes from the validator feedback itself versus simply giving the model a second attempt. This is important to call out as a deliberate design choice.

Also add: `check_validator_determinism.py` is used to verify the W3C validator itself is stable before trusting any before/after deltas, worth mentioning as a validity check.

4.5 Cost Metric

Define how you operationalise "cost" — API pricing per token, latency, or total tokens consumed per run. This addresses the supervisor's question about RQ2's cost dimension.

Change: Clarify that cost is proxied via generation latency and token counts (not API pricing), since all models run locally through Ollama. The results schema captures `gen_time_s`, `prompt_eval_count`, and `eval_count` per iteration. Note explicitly that no cloud/API-based model baseline is included in this pipeline (as stated in the repo's Scope section) — this is a scope limitation to acknowledge.

5 Experimental Setup (1 – 1.5 pages)

Describe the practical setup: hardware, software versions, Docker configuration, Ollama setup, which specific model versions/quantisations you used, and how you controlled for variability between runs. Include any pilot/exploratory results that informed your final setup (you already have some initial results with Gemma 4, Qwen 3, DeepSeek R1).

Change: Add mention of repeated trials per (model, prompt) combination, which enables variance estimation and confidence intervals rather than relying on single-run results. Also mention the `was_cleaned` flag, the pipeline strips markdown fences/preamble before validation, so formatting slip-ups are never counted as HTML errors (important for validity of the before/after comparison).

6 Results (3 – 4 pages)

6.1 RQ1 — Guardrail Effectiveness

Report error/warning counts before and after the guardrail, per model and per prompt. Show whether the guardrail improves semantic correctness — and be careful to distinguish between W3C validation errors and semantic correctness (flag this distinction explicitly here).

Change: Results now need to be broken down by condition (`feedback` vs `blind`) as well as by model and difficulty tier, not just before/after. The statistical analysis uses paired Wilcoxon tests and 95% confidence intervals, which should be stated. Also add error category breakdown as a dimension (missing-doctype, stray-tag, disallowed-attribute, etc.).

6.2 RQ2 — Cost-Efficiency

Compare smaller vs. larger models on quality metrics at various iteration counts. Show the quality-vs-cost tradeoff curve. Establish a baseline for large API-based models (or compare against a cited benchmark) as your supervisor suggested.

Change: Reframe this — the comparison is smaller vs. larger local models only, not local vs. cloud. Add the reasoning vs. non-reasoning dimension as a key finding here, since it turns out to be more predictive of success than parameter count alone. The scatter plots (model size vs. cost/quality, coloured by native thinking mode) directly address this.

6.3 Iteration Count Analysis

When does the loop converge? Is there a diminishing return after N iterations? This is where you answer the "optimal number of iterations" question.

Change: No major structural change, but the analysis now comes from `stats.py`'s iteration-cutoff analysis, which is worth naming explicitly as the source of the convergence findings.

7 Discussion (2 – 3 pages)

8 Conclusion (0.5 – 1 page)

References

- [1] Alexandra Souly, Javier Rando, Ed Chapman, Xander Davies, Burak Hasircioglu, Ezzeldin Shereen, Carlos Mougán, Vasilios Mavroudis, Erik Jones, Chris Hicks, Nicholas Carlini, Yarin Gal, and Robert Kirk. Poisoning attacks on llms require a near-constant number of poison samples, 2025. URL <https://arxiv.org/abs/2510.07192>.
- [2] Florian Tambon, Arghavan Moradi-Dakhel, Amin Nikanjam, Foutse Khomh, Michel Desmarais, and Giuliano Antoniol. Bugs in large language models generated code: an empirical study. *Empirical Software Engineering*, 30, 02 2025. doi: 10.1007/s10664-025-10614-4.
- [3] Shihan Dou, Haoxiang Jia, Shenxi Wu, Huiyuan Zheng, Muling Wu, Yunbo Tao, Ming Zhang, Mingxu Chai, Jessica Fan, Zhiheng Xi, Rui Zheng, Yueming Wu, Ming Wen, Tao Gui, Xipeng Qiu, and Xuanjing Huang. What is wrong with your code generated by large language models? an extensive study. *Science China Information Sciences*, 69, 01 2026. doi: 10.1007/s11432-025-4632-8.
- [4] Ravin Ravi, Dylan Bradshaw, Stefano Ruberto, Gunel Jahangirova, and Valerio Terragni. Llmloop: Improving llm-generated code and tests through automated iterative feedback loops. 07 2025. doi: 10.1109/ICSME64153.2025.00109.
- [5] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 03 2023.
- [6] Tristan Coignion, Clément Quinton, and Romain Rouvoy. A performance study of llm-generated code on leetcode, 07 2024.

Appendix

Full prompt dataset

Extended results tables

Flowchart (if not in main body)

Human evaluation rubric (if used)

Project timeline

- Full prompt dataset
- Extended results tables
- Flowchart (if not in main body)
- Human evaluation rubric (if used)
- Project timeline — your supervisor asked for this; an appendix is a natural place for it if it doesn't fit in the main body